Exercise Sheet 12

The Deep Archive

House rules.

- First: `pip install -r requirements.txt` inside the lesson folder (venv active). Only Exercise 7 needs it (pandas).
- One file per exercise in `my_work/`, run from the lesson folder: `python my_work/ex1_first_join.py`.
- Same opener as last week (type it):

```
import sqlite3

con = sqlite3.connect("data/guild.db")
```

- The tables: **heroes** (hero_id, name, class, kin, level, …), **quests** (quest_id, title, region, monster, difficulty, days, reward, outcome, paid, …), **party** (quest_id, hero_id, loot_share: one row per hero per quest), **regions** (region, danger, warden, travel_days). The map in `regions` has a hole in it.
- Give every table a short alias (`FROM party AS p`) and write every column as `p.hero_id`. It saves you from Exercise 1's most common error.
- Hand in: commit and push to `lesson-12`, then open a pull request. Tell Lars which JOIN surprised you.

Exercise 1 The first JOIN

10 min · `ex1_first_join.py`

> **Raven from Captain Siv:** *"The party table says H072 went on Q0001. I don't know who H072 is. Nobody knows who H072 is. Put the names in."*

(a) `FROM party AS p JOIN heroes AS h ON p.hero_id = h.hero_id`: name, class and loot share of everyone on quest `'Q0002'`, biggest share first, then by name. Empty line.

(b) Count the rows of `party`, then the rows of the join. Print both: they must be equal (every hero_id found its hero). Empty line.

(c) Wren Hawkridge: `WHERE h.name = 'Wren Hawkridge'`, her number of quests and the sum of her loot shares.

EXPECTED OUTPUT

```
THE PARTY ON Q0002
Wren Hawkridge      Ranger     220.00 cr
Idony Nightingale   Warrior    110.00 cr
Jasper Ironfoot     Warrior    110.00 cr
Ninian Underhill    Rogue      110.00 cr

party rows: 1194, after the join: 1194

Wren Hawkridge: 16 quests, 2,289.67 cr in loot
```

Hint: A JOIN is XLOOKUP for every row at once: `ON` says which column matches which. Wren got a double share on Q0002: the highest-level hero in a party leads it, and the leader takes two shares.

Exercise 2 Three tables

10 min · `ex2_three_tables.py`

> **Decree from Queen Aldara:** *"A Bard sang at court last night about 'the hardest quests in the realm'. Captain Siv coughed. I would like the numbers."*

(a) Join party, heroes **and** quests (two JOINs, each with its own ON). Per class: trips, `ROUND(AVG(q.difficulty), 2)`, total loot. Easiest first. Empty line.

(b) Wren Hawkridge's three biggest loot shares: quest_id, monster, region, share.

EXPECTED OUTPUT

```
CLASS     TRIPS  AVG DIFFICULTY        LOOT
Bard         84            2.95    9,390.75
Cleric      131            3.24   14,445.93
Rogue       176            3.32   19,524.84
Mage        224            3.39   26,730.00
Ranger      215            3.46   27,188.02
Warrior     364            3.49   48,870.51

WREN HAWKRIDGE'S THREE BIGGEST SHARES
Q0340  Bandits        Saltmarsh        295.00 cr
Q0093  Goblins        Northern Wilds   280.00 cr
Q0326  Giant Spiders  Ironhills        273.33 cr
```

Hint: Same table as lesson 10, Exercise 7, from the other direction: there it was two merges, here two JOINs. The Bard's song was, with respect, about the easiest quests in the realm.

Exercise 3 The missing region

10 min · `ex3_the_missing_region.py`

> **Raven from Orrin of the Clockworks:** *"The regions table has no Thornwood. Its warden retired to keep bees and nobody updated the map. If you JOIN quests to regions, Thornwood falls through the hole. Please don't let it."*

(a) Count `quests JOIN regions` and `quests LEFT JOIN regions` (both `ON q.region = r.region`). Print both counts. Empty line.

(b) With the LEFT JOIN, per `q.region`: `COALESCE(r.warden, '(no warden)')` and the number of quests, ordered by region. Empty line.

(c) Only the quests the map doesn't know: `WHERE r.region IS NULL`. How many, and how many crowns were paid there?

EXPECTED OUTPUT

```
INNER JOIN: 310 quests
LEFT JOIN:  400 quests

Greyfen        Warden Alder Pike          68
Ironhills      Warden Hild Stonebridge    85
Northern Wilds Warden Corwin Northwind    75
Saltmarsh      Warden Nerys Saltmarsh     82
Thornwood      (no warden)                90

Quests in regions the map doesn't know: 90 (32,780 cr paid)
```

Hint: LEFT JOIN keeps every row of the **left** table and fills the right side with NULL where nothing matched: `how="left"` from lesson 10. `COALESCE(a, b)` is IFERROR: a, unless it's NULL, then b.

Exercise 4 CASE WHEN

10 min · `ex4_case_when.py`

> **Raven from Quartermistress Pell:** *"Two tables for the Charter, please. The success rate by difficulty, with stars, because the Queen likes stars. And the success rate for Small, Fair and Rich notices: under 400 cr, 400 and up, 1,000 and up."*

(a) Per difficulty: count, and the success rate `SUM(CASE WHEN outcome = 'Success' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)`, 1 dp. Print the stars with `'*' * difficulty` in Python. Empty line.

(b) A band column: `CASE WHEN reward >= 1000 THEN 'Rich' WHEN reward >= 400 THEN 'Fair' ELSE 'Small' END AS band`. Group by band; count and success rate; order by `MIN(reward)`.

EXPECTED OUTPUT

```
SUCCESS BY DIFFICULTY
1 *       34 quests   88.2%
2 **     110 quests   74.5%
3 ***    128 quests   62.5%
4 ****    88 quests   56.8%
5 *****   40 quests   42.5%

SUCCESS BY SIZE OF NOTICE
Small   165 quests   72.1%
Fair    191 quests   63.4%
Rich     44 quests   43.2%
```

Hint: CASE is a nested IF: `=IF(r>=1000, "Rich", IF(r>=400, "Fair", "Small"))`. The first true WHEN wins, so the biggest threshold goes first. The Queen will want the 88 % and the 42 %.

Exercise 5 Questions inside questions

10 min · `ex5_questions_inside.py`

> **Raven from Captain Siv:** *"Three quick ones. Who's above the average level? Which notices were generous for their monster? And who's never once run away? I want to buy them a drink."*

(a) Print the average level (2 dp). Then count and average the heroes `WHERE level > (SELECT AVG(level) FROM heroes)`. Empty line.

(b) Quests whose reward is above **their own monster's** average: the inner query gets its own alias and `WHERE q2.monster = q.monster`. Count them per monster, most first. Empty line.

(c) Heroes whose hero_id is `NOT IN` the list of hero_ids on Fled quests (the inner query is a JOIN). How many?

EXPECTED OUTPUT

```
Average level: 7.97
Heroes above it: 62, averaging level 11.2

QUESTS PAYING ABOVE THEIR MONSTER'S AVERAGE NOTICE
Goblins          37
Wolves           31
Bandits          30
Ghouls           30
Giant Rats       19
Giant Spiders    18
Trolls           11
Wyrm              3

HEROES WHO HAVE NEVER FLED
21 of 120
```

Hint: A subquery in brackets is answered first and its answer dropped into the outer query, like a helper cell. In (b) the inner query runs again for each row, with that row's monster. Siv's round is 21 drinks; Pell says it comes out of Siv's own purse.

Exercise 6 WITH, and asking with ?

10 min · `ex6_with_and_params.py`

> **Raven from Orrin:** *"Two habits that will save your life. A WITH block, so a big query reads like a recipe. And never, ever glue a value into SQL with an f-string. Dalvric did. Ask him about 'Biscuit's Lair' (you can't; he sailed west, partly because of it)."*

(a) `WITH loot AS (SELECT hero_id, COUNT(*) AS trips, SUM(loot_share) AS total FROM party GROUP BY hero_id)`, then join `loot` to heroes and print the top five, ranked with `enumerate(..., start=1)`. Empty line.

(b) Write `region_report(region)`: one query with `WHERE region = ?`, run as `con.execute(sql, (region,))`, printing quests, wins and crowns paid. Wrap the sums in `COALESCE(..., 0)`.

(c) Call it for Greyfen, Thornwood, Northern Wilds and `"Biscuit's Lair"`.

EXPECTED OUTPUT

```
THE LOOT LEADERBOARD
1. Jasper Ironfoot     Warrior   23 trips   4,428.68 cr
2. Benedek Saltmarsh   Warrior   18 trips   3,833.34 cr
3. Silas Goldenhand    Warrior   17 trips   3,502.33 cr
4. Wren Ashdown        Ranger    20 trips   3,460.14 cr
5. Oriel Quickwater    Warrior   21 trips   3,228.82 cr

Greyfen          68 quests   38 won   18,150 cr
Thornwood        90 quests   64 won   32,780 cr
Northern Wilds   75 quests   48 won   33,600 cr
Biscuit's Lair    0 quests    0 won        0 cr
```

Hint: With `?`, sqlite3 passes the value separately, so the apostrophe in Biscuit's Lair is just a letter. Glued in with an f-string, it would end the quote early and break the query, or worse. `(region,)` needs the comma: that's what makes it a tuple.

Exercise 7 From the Vault into pandas STRETCH

15 min · `ex7_into_pandas.py`

> **Raven from Quartermistress Pell:** *"SQL fetches, pandas arranges. Orrin says they're friends. Show me: every trip with its class and outcome, straight out of the Vault into a DataFrame, and then the tables I like."*

Add `import pandas as pd` and `pd.set_option("display.width", 120)` to the opener.

(a) `trips = pd.read_sql_query(sql, con)` with a three-table JOIN selecting `h.class, q.region, q.difficulty, q.outcome, p.loot_share`. Print the shape and the first three rows. Empty line.

(b) Success rate per class: `pd.crosstab(..., normalize="index") * 100`, 1 dp, sorted by Success, best first. Empty line.

(c) Loot by region (down) and class (across): a `pivot_table` with `aggfunc="sum"`, rounded to 0 dp.

EXPECTED OUTPUT

```
(1194, 5)
     class     region  difficulty  outcome  loot_share
0   Cleric  Thornwood           1  Success       260.0
1  Warrior  Saltmarsh           4  Success       110.0
2  Warrior  Saltmarsh           4  Success       110.0
```

```
outcome  Failed  Fled  Success
class
Mage       12.9  21.4     65.6
Warrior    16.2  22.3     61.5
Ranger     20.0  19.1     60.9
Bard       11.9  27.4     60.7
Rogue      15.3  25.0     59.7
Cleric     18.3  26.0     55.7

class             Bard  Cleric    Mage  Ranger   Rogue  Warrior
region
Greyfen         1156.0  1398.0  3097.0  3505.0  2261.0   6735.0
Ironhills       2517.0  3333.0  7897.0  8101.0  2683.0  10199.0
Northern Wilds  1701.0  3497.0  5543.0  6517.0  5803.0  10538.0
Saltmarsh       1944.0  3049.0  3652.0  4082.0  3377.0  10785.0
Thornwood       2072.0  3168.0  6541.0  4983.0  5401.0  10614.0
```

Hint: `read_sql_query` turns the rows into a DataFrame with the column names from your SELECT. The Bards win as often as anyone; they're just sent on the easy ones.

Bug hunt Pages from the Grimoire, page 12

15 min · `bughunt/deep_archive.py`

Wrapped round a candle stub on the Deep Archive's oldest shelf: every region's warden, quests, heroes sent and crowns paid, for the Queen's Midsummer audience. **Three** bugs: one crashes (an "ambiguous" column name), one quietly loses a whole region, one gets the crowns subtly wrong (Thornwood paid 32,780 cr in Exercise 3). One commit per bug, each message explaining what was wrong and why.

EXPECTED OUTPUT ONCE FIXED

```
THE DEEP ARCHIVE: REGIONS
REGION         WARDEN                    QUESTS  HEROES    CROWNS PAID
Ironhills      Warden Hild Stonebridge       85     245      34,730 cr
Northern Wilds Warden Corwin Northwind       75     241      33,600 cr
Thornwood      (no warden)                   90     258      32,780 cr
Saltmarsh      Warden Nerys Saltmarsh        82     235      26,890 cr
Greyfen        Warden Alder Pike             68     215      18,150 cr
```

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.